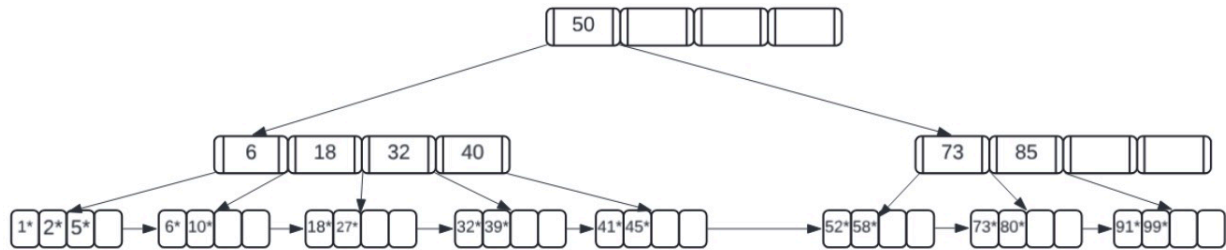


10.1

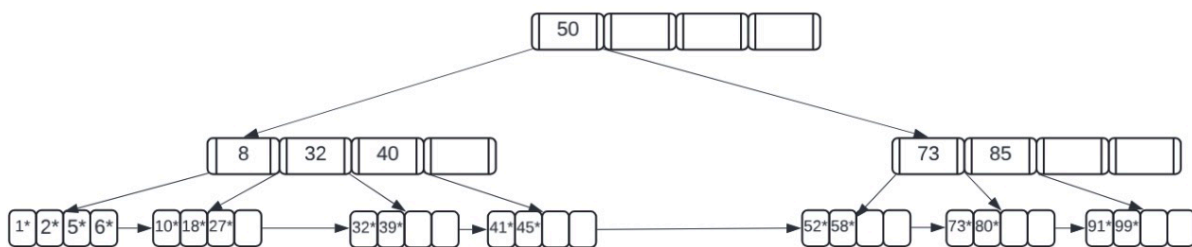
2)

The insertion will require 5 page writes, 4 page reads.

3)



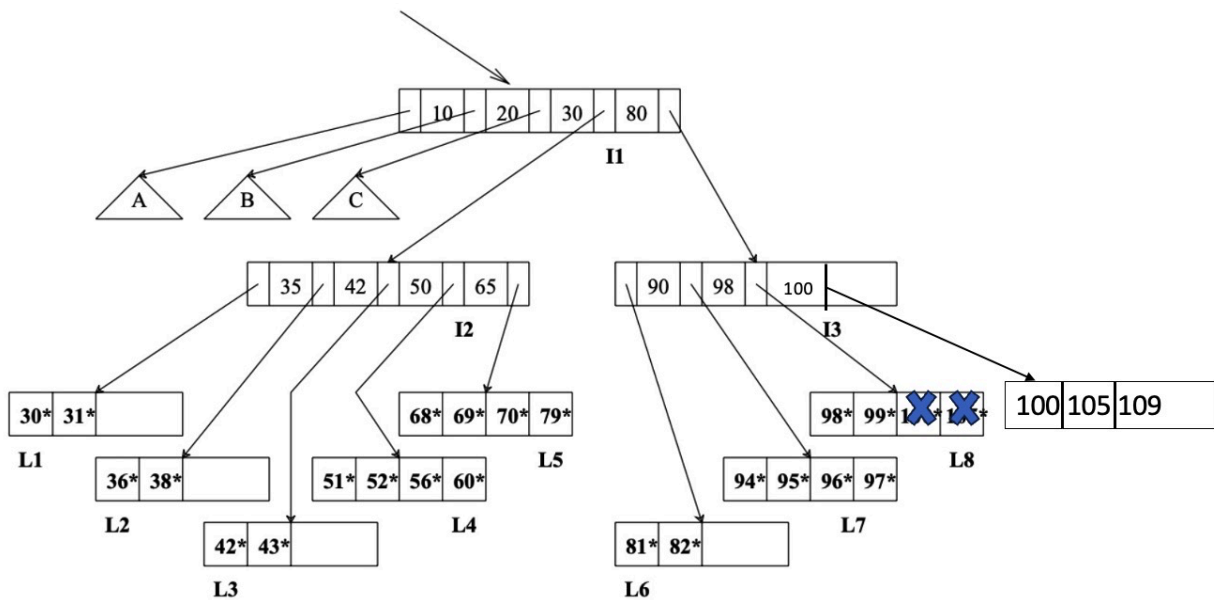
4)



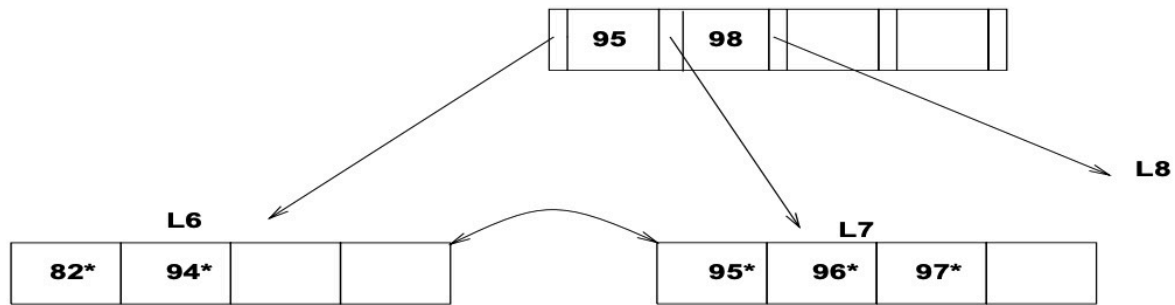
10.2

1) I1 I2 L2-L8

2)



3)



4)

Any search key in the range [65..79] would increase the height of the tree.

10.4 As long as reasonable

- 1) for instance, page exceed the page limitation after insertion
- 2) for instance, deleting the value causes underflow
- 3) for instance, underflow while redistribution is not available

11.3

1)

Global depth shows how many bits are used to index the directory. If we add more data and the directory gets full, we need to increase the global depth to make room for more buckets.

Local depth is specific to each bucket. It tells us how many bits are being used for that particular bucket to distinguish its entries from others. When a bucket fills up and has to split, local depth helps decide whether just the bucket splits or if the whole directory needs to double.

2)

Two buckets. This is because when the directory is doubled, one of the buckets must have split causing a directory entry to point to each of these two new buckets.

If an entry is then deleted from one of these buckets, a merge may occur, but this depends on the deletion algorithm. If we could merge two buckets whenever it is possible to do so then the directory size decreases after the deletion.

3)

No. If the directory is not in memory it needs to be fetched from the disk which may require more than one disk access depending upon the size of the directory. Then the required bucket has to be brought into the memory. Also, another disk access is possibly required for fetching the actual data record.

4)

The size of the directories would get large, and most of the buckets getting pointed from

those directories would be empty. As a result, many buckets will get frequently split, increasing the directory size. A skewed distribution would result in more created and unused Buckets, leading to very low data utilization.

12.2

- 1) Access the sorted file for R directly
- 2) Use the linear hashed index
- 3) Use a clustered B+ tree index
- 4) Access the sorted file for R directly

12.4

- 1)
 - a. The index matches the given selection condition. The primary conjunct is Sailors.sid<50000.
 - b. The index matches the given selection condition. The primary conjunct is Sailors.sid=50000.
- 2)
 - a. No match, as range queries cannot be applied to hash indexes.
 - b. The index matches the given selection condition. The primary conjunct is Sailors.sid=50000.
- 3)
 - a. The index matches the given selection condition. The primary conjuncts are Sailors.sid<50000 and Sailors.age=21
 - b. The index matches the given selection condition. The primary conjuncts are Sailors.sid=50000 and Sailors.age>21.
 - c. The index matches the given selection condition. The primary conjunct is Sailors.sid=50000.
 - d. No match, as there is no condition on sid
- 4)
 - a. The index matches the given selection condition. The primary conjuncts are Sailors.sid=50000 and Sailors.age=21.
 - b. No match. No able to match range
 - c. The index matches the given selection statement. The primary conjunct is Sailors.sid=50000
 - d. No. Cannot match without sid.

16.2

1. If the transaction T2 performed W(Y) before T1 performed R(Y), and then T2 aborted, the value read by T1 would be invalid and the abort would be cascaded to T1 (i.e. T1 would also have to abort).

2. Strict 2PL would require T2 to obtain an exclusive lock on Y before writing to it. This lock would have to be held until T2 committed or aborted; this would block T1 from reading Y until T2 was finished, thus there would be no interference.

3. Strict 2PL is popular for many reasons. One reason is that it ensures only 'safe' interleaving of transactions so that transactions are recoverable, avoid cascading aborts, etc. Another reason is that strict 2PL is very simple and easy to implement. The lock manager only needs to provide a lookup for exclusive locks and an atomic locking mechanism (such as with a semaphore).

16.4

1)

Lock Thrashing occurs when system performance (throughput) decreases with increasing load. It happens due to the contention of locks. Transactions waste time on lock waits.

2)

At the beginning, with the increase of transactions, throughput will increase, but the increase will stop at the thrashing point, after that point the throughput will drop with the increasing number of transactions.

3)

The throughput will increase with the increasing number of concurrent transactions.

4)

Three ways of tuning your system to increase transaction throughput:

- a. Locking the smallest sized objects possible (reducing the likelihood that two transactions need the same lock).
- b. By reducing the time that transaction hold locks
- c. By reducing hot spots. A hot spot is a database object that is frequently accessed and modified, and causes a lot of blocking delays.

16.8

1) READ UNCOMMITTED. Because we are inserting a new row in the table Catalog, we do not need any lock on the existing rows.

2) READ COMMITTED. Because we are updating one existing row in the table Catalog, we need an exclusive lock on the row which we are updating.

3) SERIALIZABLE To prevent other transactions from inserting or updating the table Catalog while we are reading from it.

4) SERIALIZABLE.